Intern Onboarding Plan — Employee Directory (HR)

Oct 28, 2026 · Mentor: @RapidM3

Welcome and goal

In 4 days you will set up Claude Code with our team configuration and build a small full-stack app on our stack: an **Employee Directory for an HR team**. Real tasks start once you finish.

The app: HR staff sign in, search employees by name, filter by department, open an employee's details, and add or edit employees. Data lives in MySQL behind a Spring Boot API, with a React frontend and Firebase login.

Your teammate does the same job on the same stack in a different context: an Inventory Tracker for a warehouse. On Day 4 you each fix a bug in the other's repo.

You are done when:

1. The app (database, API and UI) runs from a fresh clone using only your README.
2. You have opened 2 pull requests, both reviewed by your mentor: your project, plus a fix in your teammate's repo.
3. You can explain every line you submitted without asking Claude to explain it for you.

Ask your mentor before you expand scope. Flag blockers within 30 minutes; don't sit on them.

Our tech stack

You touch every layer of our stack. Each row shows what that layer does in your Employee Directory.

Layer	Technology	In your app
Database	MySQL 8, SQL	`department`, `employee` and `employee_change` tables
Backend API	Java 17+, Spring Boot, Spring Web, Spring Data JPA, Bean Validation	REST endpoints for employees and departments
Testing	JUnit, Spring Boot Test	Proves the API rules work

Layer	Technology	In your app
Frontend	React (JS/TS), Vite, CSS	Directory list, search, filter, details, add/edit form
Identity	Firebase Authentication	HR staff sign in before seeing any data
Tooling	Git, feature branches, pull requests	Every change is reviewed
AI assistant	Claude Code + team setup	Explore, plan, implement, verify

Key idea: hiding a page in React is not security. Only the backend can enforce access, by verifying the Firebase ID token on every protected request.

How to use Claude Code

Treat Claude as a fast pair programmer whose every suggestion you check. You own the correctness, security and quality of what you submit.

The 4 rules in our CLAUDE.md (Claude follows them; you should too)

1. **Think before coding.** State assumptions and ask when something is unclear.
2. **Simplicity first.** Write the minimum code that solves the task, with no speculative features.
3. **Surgical changes.** Touch only what the task needs and match the existing style.
4. **Goal-driven execution.** Define a check (test, run, query), then loop until it passes.

Plugins and when to use them

Plugin / skill	Use it for
superpowers: `brainstorming`	Before designing any feature or component
superpowers: `writing-plans`	Turning an agreed design into steps you review
superpowers: `test-driven-development`	Writing a failing test first, then the code
superpowers: `systematic-debugging`	Any bug: reproduce, gather evidence, then fix
superpowers: `verification-before-completion`	Before you say anything is done
`/code-review`	Reviewing your own diff before opening a PR

Plugin / skill	Use it for
code-simplifier	Trimming code Claude over-built
context7	Current docs for React, Firebase, Spring, so Claude doesn't invent APIs
playwright	Clicking through your UI in a real browser
frontend-design	UI layout and styling decisions
explanatory output style	Claude explains its choices as it works. Keep it on while learning

Off-limits until your mentor says otherwise: ruflo / claude-flow swarms and multi-agent commands, `ralph-loop`, the Atlassian and other connected-account tools, and deploying anywhere other than your own machine, touching a shared or production database, or pushing to `main`.

Prompt pattern for every task

```
Explore this repo and explain <area>. Do not edit files.
Then propose a small plan: files affected, assumptions, acceptance criteria, tests.
Wait for my approval before implementing.
```

Day 1: Setup, stack tour and design

Goal: Claude Code working, every layer of the stack running locally, and an approved design for your app.

Morning: environment and Claude Code warm-up

- [ ] Install JDK 17+, Maven, MySQL 8 (local), Node.js LTS and Git. Create your GitHub repo `employee-directory` with `api/` and `web/` folders.
- [ ] Warm-up: in any existing team repo, ask Claude to explain the folder structure, entry point and tests **without editing**. Write 5 things you learned in your log.
- [ ] Use `context7` to pull current Spring Boot, React and Firebase Auth docs into a session. Note one API Claude got wrong before it checked the docs, if any.

Afternoon: stack tour and design

- [ ] Generate the API with Spring Initializr (Web, Data JPA, Validation, MySQL Driver) and the frontend with Vite (React). Run both empty apps.
- [ ] Create a **personal dev** Firebase project and enable Email/Password sign-in.

- [ ] Run `superpowers:brainstorming` with this prompt, then draw the ER diagram and component tree yourself:

```
Help me design a small full-stack Employee Directory for an HR team.
Data: departments, employees (name, email, job title, department, hire date, status),
and an employee_change log recording department or job-title changes.
Propose the MySQL tables, keys, constraints and indexes, the REST endpoints,
and a React component tree with search, department filter, details view and
add/edit form, behind Firebase email/password login. List assumptions and
trade-offs. Do not write code or run SQL. Wait for my approval.
```

End of day check-in (15 min with mentor): show the ER diagram, the endpoint list and the component tree. Get approval before Day 2.

Day 2: Database and API

Goal: the database builds from scripts, and a tested, validated API serves it.

Morning: MySQL

- [ ] Create a feature branch: `git checkout -b feat/api`.
- [ ] Write `db/schema.sql` with `department`, `employee` and `employee_change`: primary keys, foreign keys, `NOT NULL` where required, a unique email, and only indexes you can justify by a query.
- [ ] Write `db/seed.sql` with fake data: 4 departments, 15+ employees, and some change rows.
- [ ] Run both scripts on a **fresh local database**. Then insert an employee with a non-existent department and a duplicate email, and confirm both are rejected.
- [ ] Write `db/queries.sql` with 5 commented queries: headcount per department, hires in the last 90 days, one employee's change history, employees by status, and departments with no employees. Check 2 results by hand.

Afternoon: Spring Boot

- [ ] Put DB credentials in environment variables or a git-ignored `application-local.properties`. Commit only an example file.
- [ ] Build one layer at a time (entity, DTOs, repository, service, controller), committing after each.
- [ ] Endpoints: `GET /employees?name=&departmentId=`, `GET /employees/{id}`, `POST /employees`, `PUT /employees/{id}`, `GET /departments`. Changing department or job title writes an `employee_change` row.

- [ ] Return `400` for invalid input, `404` for unknown IDs and `409` for a duplicate email, with a clear JSON error body.
- [ ] Use `superpowers:test-driven-development`: a failing JUnit test per rule above, then make it pass. Run `mvn test` yourself.

End of day check-in: rebuild the DB from scripts, run `mvn test`, and show a curl request for each error case.

Day 3: React UI and Firebase login

Goal: signed-in HR staff can use the whole directory in the browser; signed-out users see nothing.

Morning: React UI on the real API

- [ ] Create a branch `feat/web`. Ask Claude to turn the approved component tree into a plan (`superpowers:writing-plans`) and review it.
- [ ] Build one component at a time, committing after each: `EmployeeList`, `EmployeeCard`, `SearchBox`, `DepartmentFilter`, `EmployeeDetails`, `EmployeeForm`.
- [ ] Fetch from your API (enable CORS for the Vite dev URL only). Search and department filter must work **together**.
- [ ] Handle the states: loading, empty results, and API errors such as a duplicate email shown on the form.
- [ ] For each component, ask Claude to explain its props, state and hooks. Write one line per component in your log.

Afternoon: Firebase Auth

- [ ] Ask Claude to explain the Firebase Auth flow and `onAuthStateChanged` **before** it writes code.
- [ ] Put the Firebase web config in `web/.env.local` (Vite `VITE_` variables), git-ignore it, and commit a `.env.example`.
- [ ] Build Register, Login and Logout. Protect every directory route; signed-out users go to Login.
- [ ] Show plain messages for wrong password, unknown user, email in use and weak password.
- [ ] Stretch: send the Firebase ID token to the API and verify it in Spring with the Firebase Admin SDK. If you skip it, be ready to explain how it would work.
- [ ] Check layout at desktop and mobile widths with `playwright` screenshots.

End of day check-in: register, sign in, search + filter, add an employee, sign out, and confirm the directory is unreachable.

Day 4: Cross-repo fix, PRs and demo

Goal: practise the real-work loop on code you didn't write (your teammate's Inventory Tracker), then ship and present your own app.

Morning: fix one bug in your teammate's Inventory Tracker

Your mentor picks one small, approved issue in the teammate's repo (for example, a wrong stock total or a missing error message).

- [ ] Clone the repo, build its database from the scripts on your **local** MySQL, and run the API and UI.
- [ ] Reproduce the bug before you change anything.
- [ ] Use `superpowers:systematic-debugging` with this prompt:

```
I have a mentor-approved bug to investigate. Inspect the relevant code and
explain
the execution path without modifying files. Given my reproduction steps, list
likely
root causes and how to tell them apart. Propose a minimal fix and a
regression test,
but wait for my approval before editing.
```

- [ ] Write a failing regression test, apply the smallest fix, and confirm all tests pass.
- [ ] Open a PR covering problem, root cause, fix and test output. Your mentor reviews it.

Afternoon: ship and present

- [ ] Finish your README: prerequisites, DB setup, env variables, Firebase steps, how to run the API, UI and tests.
- [ ] Rebuild the DB from scripts one last time, run `mvn test`, and run your manual test checklist.
- [ ] Run `superpowers:verification-before-completion` and `/code-review` on the full diff, then open your PR.
- [ ] Your mentor reviews your PR. Address every comment before the final demo.
- [ ] Give your final demo to your mentor (see Final demo below).

Deliverables, review and rules

Deliverables

- [ ] `employee-directory` repo: `db/` scripts, `api/` and `web/`, all running from the README.

- [ ] `db/schema.sql`, `db/seed.sql` and `db/queries.sql` (5 commented queries) that run on a clean MySQL database, plus an ER diagram.
- [ ] API with validation and 400/404/409 handling; JUnit tests runnable with `mvn test`.
- [ ] React UI with search + department filter, details, add/edit form, loading/empty/error states, and Firebase login protecting every page.
- [ ] Manual test checklist with results: filter combinations, empty results, duplicate email, mobile width, register, wrong password, logout, protected page while signed out.
- [ ] Mentor-reviewed PR fixing one bug in your teammate's Inventory Tracker, with a regression test.
- [ ] A short Claude Code note: prompts you used, and at least one suggestion you accepted, changed or rejected, and why.

Final demo (end of Day 4, 30 min with your mentor)

Run everything locally from a fresh clone and walk your mentor through:

1. Rebuild the database from your scripts, then start the API and UI (2 min).
2. Live walkthrough: register, sign in, search + department filter, open details, add and edit an employee, trigger a duplicate-email error, sign out (8 min).
3. Show `mvn test` passing and your manual test checklist results (3 min).
4. Walk through your PR and one design decision you would defend (5 min).
5. Your fix in the Inventory Tracker: the bug, root cause and regression test (5 min).
6. Questions (7 min).

Be ready to answer:

1. Trace one request from the React search box to MySQL and back.
2. Why separate controller, service and repository? What is an entity versus a DTO?
3. What do your foreign keys and unique email prevent? Which index did you add, and why?
4. How do search and department filter combine, in the UI and in the API?
5. How does the UI know a user is signed in, and why is that not enough to protect the API?
6. Which Firebase config values are safe in a web client, and which secrets are not?
7. In your teammate's repo: what was the root cause, and what does your regression test prove?
8. Which Claude suggestion did you reject or change, and why?

Rules

- Never paste passwords, tokens, private keys or real customer or employee data into Claude.

- Never commit `.env` files, DB passwords or credentials.
- Only run SQL against your local database. No shared or production databases.
- Run, test and deploy everything locally. Nothing goes to a shared or cloud environment without mentor approval.
- Don't disable tests or weaken validation to make a build pass.
- Don't report a test as passing unless you ran it and read the output.

Daily log (fill in at the end of each day)

Day	What I completed	What I verified / learned	Claude suggestion I changed or rejected
1			
2			
3			
4			